Evidence Index

CHANGE PLAN

CPS 731: Software engineering 1

Team 16

Evidence Index:

Team Members:

- Saniya Shaikh - 501275613
- David Matan - 501130510
- Sean Rodriguez - 501257532
- Sean Huang -
- Jaydan George -

Repository link: https://github.com/David-R2/CPS-731-Booking_App
Repository evidence: none linked; this plan is self-contained in this PDF, so no commit ID applies.

Rubric criterion	Evidence	Marks	Section

Requirements and acceptance examples	Requirements table and acceptance criteria	3	Section 1
Design and trade-offs	API, data models, decision and trade-offs and component change	3	Section 2
Verification plan	Test cases and expected results.	2	Section 3
Work allocation and dependencies	Owners, dependencies, review and integration order.	2	Section 4

Requirement Table

Assumptions and shared rules

- Requests are processed sequentially; no concurrent requests are made.
- Seeded users and resources are available for acceptance testing.
- The controlled clock is set before each test.
- The acting user is identified by the `X-User-ID` request header.
- Timestamps use Toronto local time in `YYYY-MM-DDTHH:mm:ss` format, with no timezone offset or `Z`.
- A booking interval includes its start and excludes its end. Therefore, bookings that touch at an endpoint do not overlap.
- Every occurrence must start strictly after the controlled current time, end after it starts, and last at most 8 hours. Exactly 8 hours is allowed.
- A cancelled booking remains available in reads, has a cancelled status, and no longer occupies its time slot.

Case ID	Requirement	Acceptance criteria
C1	Create weekly occurrences for a recurring request. The recurrence count is from 1 to 12.	**Succeed:** For an existing resource and a recurrence count from **1 through 12**, create the requested number of weekly occurrences on the same weekday and at the same local time. A count of 1 creates one occurrence. **Fail:** Reject a count below 1 or above 12, or a resource ID that does not exist. Create no series or occurrences when the request is rejected.
C2	Make the series and its events belong to the acting user, identified by `X-User-ID`. The client cannot designate another owner in the request body.	**Succeed:** If `X-User-ID` identifies an existing user, assign that user as the owner of the series and every occurrence. **Fail:** Reject `owner_id` in the request body as an unknown field. Reject an unknown `X-User-ID`. A rejected request creates no records.
C3	Validate each occurrence's start time, end time, and duration.	**Succeed:** Accept an occurrence only when its start is strictly after the controlled current time, its end is after its start, and its duration is at most 8 hours. **Fail:** Reject an occurrence that starts at or before the controlled current time, ends at or before its start, or lasts more than 8 hours. For example, reject 13:00–22:00.

C4	Reserve each resource exclusively, using start-inclusive and end-exclusive intervals.	**Succeed:** With the controlled clock set before the bookings, accept 13:00–14:00 and 14:00–15:00 on the same resource. These intervals touch but do not overlap. **Fail:**Reject a request for 13:59–14:30 on that resource because it overlaps the 13:00–14:00 booking. Leave the schedule unchanged after rejection.
C5	Check every requested occurrence against existing uncancelled bookings. Create the entire series only if every occurrence is available.	**Succeed:** Check all requested occurrences against uncancelled individual bookings and occurrences from other series on the same resource. If every occurrence is available, create the full series and all its occurrence bookings. **Fail:** If any occurrence overlaps an existing uncancelled booking, reject the request without creating the series or any occurrence bookings. Preserve all existing bookings.
C6	Persist accepted series and occurrences. Make occurrences available through booking reads and expose their series membership through the series interface.	**Succeed:** Store the accepted series and every occurrence. Booking retrieval/listing returns the occurrences, and the series interface exposes the series and its membership. All occurrences have the same `series_id`; the series can be viewed from its first occurrence through its last. The records remain available after the application restarts. **Fail:** Reject as unsuccessful any implementation where the series or an occurrence is missing from storage or required reads, or is lost after restart.
C7	Allow the owner to cancel one occurrence without cancelling others. An occurrence can only be cancelled strictly before its start.	**Succeed:** If the owner cancels an occurrence at 10:00 when it starts at 11:00, mark that occurrence cancelled and keep it available in reads. Leave the other occurrences unchanged. **Fail:** Reject cancellation by a non-owner or at or after the occurrence's start. Leave the occurrence unchanged after rejection.
C8	Allow the owner to cancel all remaining uncancelled future occurrences in a series. Preserve ongoing and past occurrences.	**Succeed:** If the controlled clock is 10:00, an owner's request cancels all uncancelled occurrences with a start strictly after 10:00. Mark them cancelled and retain them in reads. Preserve occurrences that have started or ended. If no uncancelled future occurrences remain, return success without changing booking state. **Fail:** Reject a non-owner's request and leave the booking state unchanged.

C9	Retain cancelled records and expose them in reads. Cancelled bookings occupy no slot. Repeated cancellation does not create further changes, and ownership checks still apply.	**Succeed:** After cancellation, the booking remains visible in reads with a cancelled status and no longer blocks its time slot. Repeating the cancellation as the owner returns success and leaves its state unchanged. **Fail:** Reject a non-owner's cancellation request, including for an already-cancelled booking. Leave the booking state unchanged. Uncancelled bookings remain scheduled and are returned as uncancelled in reads.
C10	Preserve single-booking operations and make their conflict checks work with recurring bookings.	**Succeed:** Accept a valid single booking for an available resource and time slot. **Fail:**Reject a single booking that overlaps an uncancelled occurrence from a recurring series or another uncancelled booking. A cancelled booking does not block the slot.
C11	Demonstrate sequential request handling and all-or-nothing series creation. Concurrent-request guarantees are outside scope.	**Succeed:** First create a booking for 13:00–14:00 on a resource. Then submit a request for 13:30–14:30 on the same resource. The first request succeeds; the second is rejected, and the first booking remains unchanged. If a series request conflicts on any occurrence, reject it without creating any part of the series. Each request is checked against the schedule resulting from earlier successful requests. **Assumption/limit:**Requests are processed sequentially. Simultaneous-request guarantees are outside the required implementation.
C12	Use the supplied controllable clock and preserve the campus-local time convention.	**Succeed:** With the controlled clock set to Monday 2026-10-05 09:00:00, accept a request starting Monday 2026-10-05 10:00:00 in `YYYY-MM-DDTHH:mm:ss` format. Schedule weekly occurrences at the same local weekday and time. Use the controlled clock for future-time checks. **Fail:** Reject `2026-10-05T10:00:00-04:00` and `2026-10-05T10:00:00Z`. Do not use the machine's actual time for acceptance decisions.

Design and Trade-offs

2. Design and Trade-offs

2.1 Proposed API

Operation	Request	Success
`POST /booking-series`	`X-User-ID` header and JSON containing `resource_id`, `start`, `end`, and integer `recurrence_count` from 1 to 12	`201`, series details and occurrence bookings
`GET /series/{booking_id}`	Positive booking ID; no identity required	`200`, all occurrences with `series_id` in chronological order
`POST /bookings/{booking_id}/cancel`	`X-User-ID`; optional `?future=true`	`200`, cancelled booking or updated series

2.2 Data Model and Component Changes

Use the existing `Booking` model and `booking` table. Add a nullable `series_id` column. Each occurrence is stored as a regular booking row; all occurrences in one series share the same `series_id`. Individual bookings have `series_id = NULL`. Each occurrence has its own unique booking ID.

The series creation request contains `resource_id`, `start`, `end`, and `recurrence_count`. The server generates `series_id`; the request does not accept it.

Leave BookingRead unchanged. BookingSeriesRead contains the booking fields plus series_id. `GET /series/{booking_id}` accepts any occurrence's booking ID and returns the series' occurrences in chronological order as a list of BookingSeriesRead objects. If the booking is not part of a series, return 409 error status. `GET /bookings` and `GET /bookings/{booking_id}` do not expose series membership.

Component	Change
`booking/main.py`	Add series creation and retrieval routes; extend the existing cancellation route to support `future=true`.
`booking/models.py`	Add nullable `series_id` to `Booking`; do not add a separate series database model or table.
`booking/schemas.py`	Add `BookingSeriesCreate` and `BookingSeriesRead` schemas. Keep `series_id` out of request schemas.
`booking/database.py`	Keep existing SQLite databases to add the nullable `series_id` column.
Tests and README	Test and document recurrence, conflicts, cancellation, persistence, transaction behavior, and concurrency limits.

Occurrences remain available through the existing `GET /bookings` and `GET /bookings/{booking_id}` operations. `GET /series/{booking_id}` returns the group and its occurrences. Records must remain available after application restart.

Occurrences are generated seven days apart at the same campus-local weekday and time. For every occurrence, the start must be strictly after the supplied clock's current time, the end must follow the start, and duration must not exceed eight hours. Overnight bookings are allowed.

Conflicts are checked against uncancelled bookings on the same resource, including individual bookings and occurrences from other series. Intervals include their start and exclude their end, so adjacent bookings are allowed. Different resources do not conflict. Existing single-booking operations must also detect conflicts with recurring occurrences.

2.3 Decisions and Alternatives

Decision	Reason and trade-off
Store one booking row per occurrence and group rows with `series_id`.	This reuses existing booking listing, retrieval, cancellation, and conflict checks. A separate `Series` table could store series metadata directly, but would add a table and relationships that can be derived from the occurrence rows.

Keep the existing cancellation route and add optional `future=true`.	Existing requests still cancel one booking by default. A separate series-cancellation route was an alternative; the chosen approach keeps both behaviors on one route.
Add `BookingSeriesRead` and `BookingSeriesCreate.`	Adds a single endpoint-specific shape for creating a series and returning its details. The create schema accepts only the recurrence inputs, so clients cannot choose `owner_id` or `series_id`. `BookingSeriesRead` exposes the generated `series_id` and the series' occurrences while leaving the existing `BookingRead` response unchanged. The trade-off is that clients cannot identify series membership from `GET/bookings` or `GET/bookings/{booking_id};` they must use the series endpoint.
Use the first occurrence's generated booking ID as `series_id`.	This avoids a separate series table or ID-generation service. Every occurrence still has its own unique booking ID.
Reject `future=true` cancel requests when the selected booking is not part of a series.	This prevents a request intended to cancel future series occurrences from being applied to a standalone booking. The booking can still be cancelled individually without `future=true`.

`X-User-ID` must identify a valid seeded user, and `resource_id` must identify an existing resource. The acting user owns the series and all its occurrences. The request body cannot set `owner_id` or `series_id`.

When the future is omitted or false, cancel only the booking identified by `booking_id.` If that booking is already cancelled, return 200 with no changes. When `future=true`, cancel all uncancelled occurrences in the booking's series whose starts are strictly after now and return 200. When `future=true`, return 422 if the selected booking has no `series_id` or all the occurrences are in the past, preserve their status. If the bookings are already canceled and in the future, return 200 and preserve their status. If future bookings are a mix of cancel and uncancelled booking, cancel the uncancelled bookings preserve the status of the canceled bookings and return 200. Only the owner may cancel; check ownership before handling the request, including retries.. Only the owner may cancel; check ownership before handling either case, including retries.

2.4 All-or-Nothing Series Creation

First validate the user, resource, recurrence count, timestamps, duration, and every occurrence's availability. Check each occurrence against existing uncancelled bookings on the same resource before inserting new rows.

If all occurrences are valid and available, insert them in one database transaction and commit once. Flush the first occurrence to obtain its generated booking ID, assign that value as the shared `series_id`, then commit the occurrence rows. A flush sends SQL within the transaction but does not commit. If an error occurs before the commit, roll back so no new occurrence rows are saved and existing bookings remain unchanged.

Tests use the supplied controllable clock and existing campus-local timestamp rules. They verify sequential requests and confirm that a conflict in any occurrence creates no new occurrence rows and leaves existing bookings unchanged. Simultaneous requests can still race between availability checks and insertion; concurrent-request guarantees are outside scope.

Verification Plan

1. Purpose and scope

This plan verifies the design for weekly recurring bookings and checks that existing single-booking behavior continues to work after the change.

The design is the source of truth for expected behavior:

- `POST /booking-series` creates weekly occurrences.
- `GET /series/{booking_id}` accepts **any occurrence's booking ID** and returns its series in chronological order.
- Each occurrence is stored as a normal booking row. Its `series_id` is the booking ID of the series' first occurrence.
- Existing `GET /bookings` and `GET /bookings/{booking_id}` responses stay unchanged; they do **not** expose `series_id`.
- `POST /bookings/{booking_id}/cancel` cancels one occurrence by default. With `future=true`, it cancels eligible future occurrences in that series.
- A `future=true` request for a standalone booking returns `422`.
- All series occurrences must be available before any are inserted. A conflict in any occurrence rejects the series without creating partial records.

The plan covers requirements C1–C12, risks of partial series creation and incorrect conflict detection, and regression of the existing single-booking API.

2. Test mechanism and controlled data

Use `pytest`, the application's API test client, and a temporary SQLite database.

For each test:

1. Create an isolated database.
2. Seed the required users and resources.

3. Set the application's controllable clock explicitly when the test depends on time.
4. Add any required existing bookings as fixtures.
5. Make API requests through the test client.
6. Check response status and JSON, then inspect the database or API reads to confirm the intended records and unchanged state.

Use Toronto local, offset-free timestamps. Unless a test specifies another clock, set it to **2026-10-01 09:00:00**. The default three-occurrence series starts **2026-11-02 10:00–11:00**, with occurrences on November 2, 9, and 16. Those dates are Mondays. Use the same resource for conflict tests and a different resource for non-conflict tests.

For every rejected request, confirm that the database contains no unintended records and that existing records remain unchanged. For successful requests, check the exact number and values of the created rows.

Proposed Failure Codes:

- **HTTP 422:** Invalid request fields, timestamp boundaries, etc.
- **HTTP 404:** Requested resource does not exist within the existing API
- **HTTP 403:** Cancellation attempted by a non-owner
- **HTTP 409:** Booking conflicts with an existing reservation that hasn't been cancelled

3. Pass criteria

A test passes only when:

- The HTTP status and response fields match the expected result.
- Successful series creation returns the requested number of occurrences, with unique booking IDs, correct owner/resource/times, and the shared series ID.
- Reads return the expected records and order.
- Rejected requests leave existing records unchanged and create no partial series.
- Cancellation retains records and changes only the eligible occurrence or occurrences.
- Single-booking requests retain their existing request/response contract and work correctly alongside recurring occurrences.

Test cases

ID	Requirement	Setup and action	Expected result
T01	C1, C2, C3, C6	As user 1, create a one-occurrence series on resource 1 for a valid future interval.	`201`. Exactly one occurrence is returned and stored. Its owner is user 1 and its `series_id` equals its own booking ID.
T02	C1, C3, C6	Create a 12-occurrence series.	`201`. Exactly 12 rows are returned and stored. IDs are unique; all rows share a `series_id`; consecutive starts are seven days apart at the same local time.
T03	C1	Request a series with `recurrence_count=0`.	`422`. No rows are created.
T04	C1	Request a series with `recurrence_count=13`.	`422`. No rows are created.
T05	C1, C3	Create a three-occurrence series starting November 2, 2026, at 10:00–11:00.	`201`. Starts are November 2, 9, and 16 at 10:00; ends are at 11:00. Returned and stored times match.
T06	C2	Create a series with `X-User-ID: 1`.	`201`. Every occurrence has `owner_id=1`. No unrelated records change.

T07	C2	Send `owner_id` in the creation body while using `X-User-ID: 1`.	`422`. No rows are created; the body cannot select the owner.
T08	C2	Create a series with an unknown user ID.	`404`. No rows are created.
T09	C1	Create a series with an unknown resource ID.	`404`. No rows are created.
T10	C3, C12	Set the clock to November 2, 2026, at 10:00 and request a first occurrence starting at exactly 10:00.	`422`. No rows are created because the start must be strictly in the future.
T11	C3	Set the clock to November 2, 2026, at 10:00 and request a first occurrence starting at 09:00.	`422`. No rows are created.
T12	C3	Request an occurrence whose end equals its start.	`422`. No rows are created.
T13	C3	Request an occurrence lasting exactly eight hours.	`201`. The occurrence is stored with the requested times.
T14	C3	Request an occurrence lasting eight hours and one minute.	`422`. No rows are created.
T15	C3	Request a valid overnight occurrence no longer than eight hours.	`201`. The overnight interval is accepted and stored accurately.

T16	C12	Submit a timestamp with a `Z` suffix, then one with a UTC offset.	Each request returns `422`; neither creates records.
T17	C12	Set the supplied clock to a fixed time and request a start after that time, even if it is before the machine's actual time.	The request is judged against the supplied clock and succeeds if otherwise valid.
T18	C4, C10	Create single bookings on the same resource for 10:00–11:00 and 11:00–12:00.	Both succeed. Two distinct rows are stored.
T19	C4, C10	Create a single booking for 10:00–11:00, then request 10:59–12:00 on the same resource.	First succeeds; second returns `409`. The first row remains unchanged; no second row is stored.
T20	C5	Create a three-occurrence series when every occurrence is available.	`201`. All three occurrences are stored.
T21	C5	Seed a standalone booking overlapping the second occurrence, then request a three-occurrence series.	`409`. No occurrence from the new series is stored; the standalone row is unchanged.
T22	C5	Seed an occurrence from another series that overlaps the third requested occurrence.	`409`. No partial series is stored; the existing series remains unchanged.

T23	C5, C11	Seed a standalone booking overlapping only the final occurrence of a new series; confirm earlier requested weeks are free.	`409`. No earlier occurrence is stored. The seeded row is unchanged.
T24	C5, C11	Create series A, then request series B so only B's second occurrence overlaps A.	A returns `201`; B returns `409`. A remains unchanged and none of B's occurrences are stored.
T25	C4, C5	Create a series on resource 1, then request an overlapping series on resource 2.	Both succeed. Different resources do not conflict.
T26	C4, C5	Create series A, then series B whose matching intervals begin exactly when A's end times occur.	Both return `201`. All six rows are stored and A remains unchanged.
T27	C5	Submit the same series request twice.	First returns `201`; second returns `409`. The database still contains only the first series.
T28	C5, C11	After any rejected series request, compare all pre-existing booking fields and IDs with their original values.	Existing rows are identical; there are no partial or extra rows from the rejected request.
T29	C6	Create a series and call `GET /bookings`.	`200`. Each occurrence appears as a separate booking record. The existing response shape is unchanged and does not include `series_id`.

T30	C6	Retrieve each occurrence using `GET /bookings/{booking_id}`.	Each returns `200` with the existing booking response fields and correct stored values. The response does not include `series_id`.
T31	C6	Create a series, then call `GET /series/{booking_id}` using the first occurrence's booking ID.	`200`. All occurrences are returned in chronological order with series membership exposed by the series response.
T32	C6	Repeat T31 using a non-first occurrence's booking ID.	`200`. It returns the same series and chronological occurrence list.
T33	C6	Call the series endpoint with a booking ID that does not exist.	`404`. No database state changes.
T34	C6	Call the series endpoint with a standalone booking's ID.	`409`, the booking is not a series. No database state changes.
T35	C6	Create a series, close the app/database connection, then reopen the app and retrieve the series and occurrences.	Records, IDs, times, owners, and cancellation states persist unchanged.
T36	C7	As the owner, cancel only the November 9 occurrence before it starts, without `future=true`.	`200`. Only that occurrence becomes cancelled; all rows remain available in reads.

T37	C7	Set the clock to exactly an occurrence's start and attempt to cancel it.	`409`. It remains uncancelled; other occurrences remain unchanged.
T38	C7	Set the clock after an occurrence has started and attempt to cancel it.	`409`. It remains unchanged.
T39	C7, C8	Attempt cancellation as a non-owner.	`403`. No occurrence changes.
T40	C8	Set the clock to November 5, 2026, and cancel the series with `future=true` as its owner.	`200`. November 9 and 16 are cancelled; November 2 is unchanged. All rows remain stored.
T41	C8	Use a series containing past, currently-started, and future occurrences; cancel with `future=true`.	`200`. Only uncancelled occurrences whose starts are strictly after the clock are cancelled. Past and started occurrences are unchanged.
T42	C8	Use `future=true` there are only cancelled future occurrences remaining.	`200`. Return the design-specified success result without changing any rows.
T43	C8	Use future = true when there are a mix of canceled and uncanceled bookings in the future.	Return 200, and cancel the uncanceled bookings, preserve the canceled booking status.

T44	C8	Use future=true when all the booking are in th past	Return 422, booking preserve its status, do not change any rows.
T45	C8	Use `future=true` on a standalone booking.	`422`. The booking remains unchanged.
T46	C9	Cancel an occurrence, then create a standalone booking for the same resource and interval.	Cancellation returns `200`; new booking succeeds. The cancelled occurrence remains stored.
T47	C9	Cancel an occurrence, then retrieve it through `GET /bookings/{booking_id}`.	`200`. It remains present with `cancelled=true` and its original fields.
T48	C9	As the owner, repeat an occurrence cancellation, including after its start time.	`200` with the unchanged cancelled record. No extra rows or state changes occur.
T49	C9	After the owner cancels an occurrence, retry as another user.	`403`. Ownership is checked even on retries; the record remains unchanged.
T50	C10	Create a valid single booking using the existing `POST /bookings` contract.	`201`. Response contains the existing booking fields; its behavior and shape are unchanged.
T51	C10	Retrieve and list single bookings using the existing GET endpoints.	`200`. Existing response fields and ordering are preserved.

T52	C10	Restart the app after creating a single booking.	The booking remains available with unchanged values.
T53	C10	Attempt a single booking overlapping an uncancelled series occurrence.	`409`. No single booking row is added; the occurrence remains unchanged.
T54	C10	Cancel a series occurrence, then create an overlapping single booking.	The single booking succeeds return `201`; both the retained cancelled occurrence and new single booking are readable.
T55	C10	Create a single booking, then request a series that overlaps it.	The series returns `409`; the single booking is unchanged and no series rows are stored.
T56	C10	Confirm ordinary single-booking request validation, unknown-resource handling, and cancellation ownership	Existing status codes, response bodies, and persistence behavior remain unchanged.

Work allocation

Work allocation and integration plan

David — Code management, coordination, and database/schema changes
Coordinate API and schema decisions across the team, implement the booking schema and database changes for `series_id`, and review integrated changes for consistency with the design. Keep the team aligned on response shapes and shared behavior.

Sean — Series creation: `POST /booking-series`
Implement recurring booking creation, including validation, weekly occurrence generation, conflict checks for every occurrence, and all-or-nothing persistence. Update the documentation for the endpoint and verify that existing `POST /bookings` behavior still works, including conflict checks against recurring occurrences.

Sean R — Series retrieval: `GET /series/{booking_id}`
Implement retrieval of a series from any occurrence's booking ID, returning occurrences in chronological order. Handle the case where the booking is not part of a series and document the endpoint. Coordinate with David and Sean on schema and response shapes.

Jaydan — Cancellation: `POST /bookings/{booking_id}/cancel`
Implement individual cancellation and the `future=true` series cancellation behavior. Enforce ownership and time rules, preserve past and ongoing occurrences, and make retries safe. Document both behaviors and coordinate response codes with the team.

Saniya — Tests and documentation
Write and maintain tests for the new and existing behaviors, including validation, conflicts, all-or-nothing creation, retrieval, cancellation, persistence, and regression coverage for single bookings. Update the project documentation to explain the complete feature and its usage. Coordinate test expectations with each implementer.

Dependencies and review order

1. **David** defines the schema and database first, and shares the agreed request and response shapes with the team.
2. **Sean** and **Sean R** implement creation and retrieval against those shared shapes. They coordinate on the series ID and occurrence representation.

3. **Jaydan** implements cancellation using the same series representation and coordinates cancellation response behavior with David.
4. **Saniya** develops tests alongside implementation, then runs the full verification suite against the integrated changes and updates the documentation.
5. **David** reviews the combined changes for integration issues; each feature owner addresses review findings before the team considers the work complete.